In [ ]:
```
# Experiment 7: Handling Outliers & Missing Values in a Dataset

PIN:241UDS0097
Aim: To preprocess a dataset by removing duplicates, detecting outliers, and
handling missing values using Pandas and Scikit-learn `SimpleImputer`.
```

In [ ]:
```
## Step 1: Import the basic libraries
```

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [ ]:
```
## Step 2: Load & analyze the dataset
Keep the CSV file in the same folder as this notebook.
```

In [3]:
```
students_data_241UDS0097 =
pd.read_csv('Experiment_7_241UDS0097_Students_Dirty_Dataset.csv')
students_data_241UDS0097.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 306 entries, 0 to 305
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Roll No     306 non-null    int64
 1   Name        306 non-null    object
 2   Branch      306 non-null    object
 3   Age         287 non-null    float64
 4   Attendance  297 non-null    float64
 5   GPA         290 non-null    float64
 6   City        293 non-null    object
dtypes: float64(3), int64(1), object(3)
memory usage: 16.9+ KB
```

In [4]:
```
students_data_241UDS0097.head()
```

Out[4]:

	Roll No	Name	Branch	Age	Attendance	GPA	City
0	1011	Meera	ECE	22.0	77.07	6.25	Kolkata
1	1012	Tanvi	Mechanical	21.0	79.27	8.01	Kolkata
2	1013	Rohan	EEE	20.0	94.14	6.57	Jaipur
3	1014	Vivaan	IT	19.0	80.29	5.63	Jaipur
4	1015	Aadhya	IT	21.0	76.34	8.40	Bangalore

In [5]:
```
students_data_241UDS0097.shape
```

Out[5]:
```
(306, 7)
```

In [ ]:
```
## Step 3: Find & remove the duplicates
```

In [6]:
```
# Checking the number of duplicated Age/City/GPA combinations
students_data_241UDS0097[['Age', 'City', 'GPA']].duplicated().sum()
```

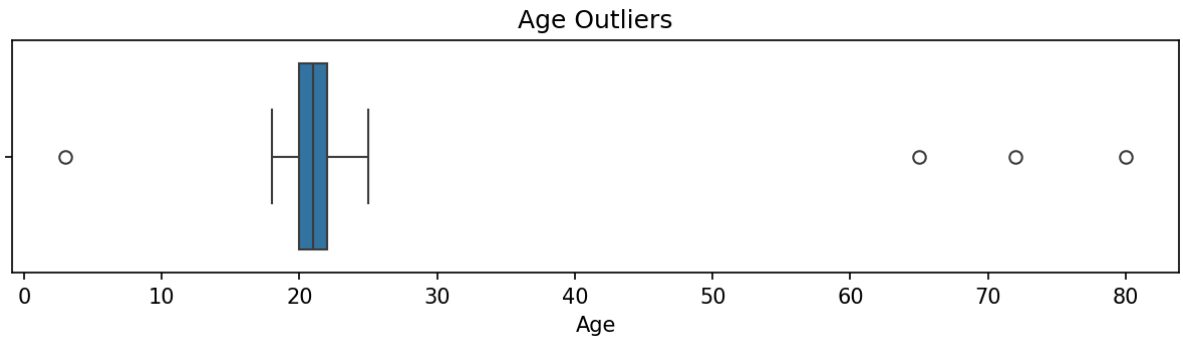
Out[6]:
```
np.int64(7)
```

In [7]:
```
# Remove fully duplicated rows
students_data_241UDS0097 = students_data_241UDS0097.drop_duplicates()
students_data_241UDS0097.shape
```

Out[7]:
```
(300, 7)
```

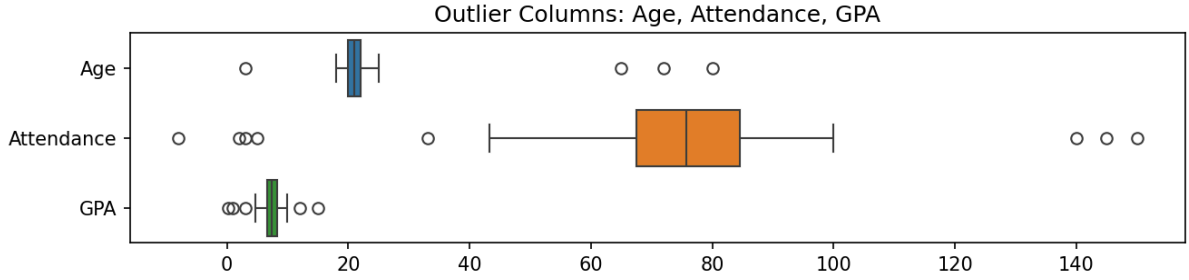
In [ ]:
```
## Step 4: Identify & remove the outliers
Use boxplots to inspect Age, Attendance, and GPA.
```

In [9]:
```
plt.figure(figsize=(10, 2), dpi=150)
sns.boxplot(data=students_data_241UDS0097['Age'], orient='h')
plt.title('Age Outliers')
plt.show()
```



In [10]:
```
# Create a list for outlier columns
outlier_columns_241UDS0097 = ['Age', 'Attendance', 'GPA']

# Visualization with boxplots
plt.figure(figsize=(10, 2), dpi=150)
sns.boxplot(data=students_data_241UDS0097[outlier_columns_241UDS0097],
orient='h')
plt.title('Outlier Columns: Age, Attendance, GPA')
plt.show()
```



In [ ]:
```
### Remove outliers with IQR ranges
```

In [11]:
```
# Calculate the first and third quartiles
Q1_241UDS0097 =
students_data_241UDS0097[outlier_columns_241UDS0097].quantile(0.25)
Q3_241UDS0097 =
students_data_241UDS0097[outlier_columns_241UDS0097].quantile(0.75)
Q1_241UDS0097, Q3_241UDS0097
```

Out[11]:
```
(Age           20.0000
 Attendance    67.5150
 GPA            6.6575
 Name: 0.25, dtype: float64,
 Age           22.0000
 Attendance    84.5650
 GPA            8.3225
 Name: 0.75, dtype: float64)
```

In [12]:
```
# IQR
IQR_241UDS0097 = Q3_241UDS0097 - Q1_241UDS0097
IQR_241UDS0097
```

Out[12]:
```
Age            2.000
Attendance    17.050
GPA            1.665
dtype: float64
```

In [13]:
```
# Find outliers using the IQR range
outlier_mask_241UDS0097 =
(students_data_241UDS0097[outlier_columns_241UDS0097] < (Q1_241UDS0097 - 1.5
* IQR_241UDS0097)) | (students_data_241UDS0097[outlier_columns_241UDS0097] >
(Q3_241UDS0097 + 1.5 * IQR_241UDS0097))
outlier_mask_241UDS0097.sum()
```

Out[13]:
```
Age           4
Attendance    8
GPA           5
dtype: int64
```

In [14]:
```
# Shape before removing outliers
students_data_241UDS0097.shape
```

Out[14]:
```
(300, 7)
```

In [15]:
```
# Keep rows that are not outliers in any of the selected numeric columns
students_data_241UDS0097 =
students_data_241UDS0097[~outlier_mask_241UDS0097.any(axis=1)].copy()

# Shape after removing outliers
students_data_241UDS0097.shape
```

Out[15]:
```
(283, 7)
```

In [16]:
```
students_data_241UDS0097.head()
```

Out[16]:

	Roll No	Name	Branch	Age	Attendance	GPA	City
0	1011	Meera	ECE	22.0	77.07	6.25	Kolkata
1	1012	Tanvi	Mechanical	21.0	79.27	8.01	Kolkata
2	1013	Rohan	EEE	20.0	94.14	6.57	Jaipur
3	1014	Vivaan	IT	19.0	80.29	5.63	Jaipur
4	1015	Aadhya	IT	21.0	76.34	8.40	Bangalore

In [ ]:
```
## Step 5: Fill the missing values
```

In [17]:
```
# Count missing values in each column
students_data_241UDS0097.isnull().sum()
```

Out[17]:
```
Roll No        0
Name           0
Branch         0
Age           17
Attendance     9
GPA           14
City          12
dtype: int64
```

In [ ]:
```
### Fill Age with the mean value using NumPy
```

In [18]:
```
students_data_241UDS0097['Age'] =
students_data_241UDS0097['Age'].fillna(np.mean(students_data_241UDS0097['Age
']))
students_data_241UDS0097['Age'].isnull().sum()
```

Out[18]:
```
np.int64(0)
```

In [ ]:
```
### Fill Attendance and GPA with median values using SimpleImputer
```

In [20]:
```
from sklearn.impute import SimpleImputer
```

In [22]:
```
numerical_imputer_241UDS0097 = SimpleImputer(strategy='median')
students_data_241UDS0097[['Attendance', 'GPA']] =
numerical_imputer_241UDS0097.fit_transform(students_data_241UDS0097[['Attend
ance', 'GPA']])
```

In [23]:
```
students_data_241UDS0097[['Attendance', 'GPA']].isnull().sum()
```

Out[23]:
```
Attendance    0
GPA           0
dtype: int64
```

In [ ]:
```
### Fill City with the most frequent value using SimpleImputer
```

In [24]:
```
students_data_241UDS0097['City'].value_counts()
```

Out[24]:
```
City
Delhi        38
Lucknow      35
Bangalore    33
Ahmedabad    30
Jaipur       27
Chennai      24
Pune         23
Mumbai       22
Hyderabad    21
Kolkata      18
Name: count, dtype: int64
```

In [26]:
```
categorical_imputer_241UDS0097 = SimpleImputer(strategy='most_frequent')
students_data_241UDS0097[['City']] =
categorical_imputer_241UDS0097.fit_transform(students_data_241UDS0097[['City
']])
```

In [27]:
```
students_data_241UDS0097['City'].value_counts()
```

Out[27]:
```
City
Delhi        50
Lucknow      35
Bangalore    33
Ahmedabad    30
Jaipur       27
Chennai      24
Pune         23
Mumbai       22
Hyderabad    21
Kolkata      18
Name: count, dtype: int64
```

In [ ]:
```
## Final check: cleaned dataset info
```

In [28]:
```
students_data_241UDS0097.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 283 entries, 0 to 299
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype  
---  ------      --------------  -----  
 0   Roll No     283 non-null    int64  
 1   Name        283 non-null    object 
 2   Branch      283 non-null    object 
 3   Age         283 non-null    float64
 4   Attendance  283 non-null    float64
 5   GPA         283 non-null    float64
 6   City        283 non-null    object 
dtypes: float64(3), int64(1), object(3)
memory usage: 17.7+ KB
```

In [29]:
```
# Confirm that no missing values remain
students_data_241UDS0097.isnull().sum()
```

Out[29]:
```
Roll No       0
Name          0
Branch        0
Age           0
Attendance    0
GPA           0
City          0
dtype: int64
```

In [ ]:
```
 Result :-
Using Pandas and Scikit-learn, duplicate rows were removed, outliers were identified using the IQR method and dropped, and missing values were filled using mean, median, and most-frequent strategies. The cleaned dataset is ready for further analysis.
```